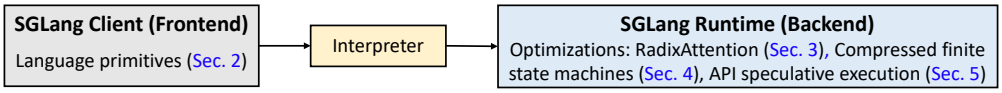


Figure 1:

SGLang:

Lianmin Zheng[2*] **Liangsheng Yin**[3] **Zhiqiang Xie**[1] **Chuyue Sun**[1] **Jeff Huang**[4]
Cody Hao Yu[5] **Shiyi Cao**[2] **Christos Kozyrakis**[1] **Ion Stoica**[2] **Joseph E. Gonzalez**[2]
Clark Barrett[1] **Ying Sheng**[1*]

[1] Stanford University [2] UC Berkeley [3] Shanghai Jiao Tong University
[4] Texas A&M University [5] Independent Researcher

Abstract

(LLM) / SGLang SGLang KV RadixAttentionSGLang 6.4× Agent JSON
https://github.com/sgl-project/sglang

1

(LLM) [35, 6, 36, 52, 46] (LLM) [41, 38] [47, 2] [30] [5] [53] [33] [56] (LLM) [57, 21]

(LLM)(LLM)(LLM)""LM Programs [4, 20] LM LM (1) LM(LLM) (2) LMLMLM

LMLM *(LLM)LM* LM Sec. 2

LM vLLM [23]TGI [16]TensorRT-LLM [34] KVSec. 3KV LM(LLM)KV JSON(LLM)Sec. 4 tokentoken

SGLang(LLM)Structured Generation Language LM Fig. 1 LM

SGLangPythonextendgenselectforkjoinSGLangPythonPythonSGLang SGLang

SGLangRadixAttentionKVKVKVKVLRUKV tokentokentokentokentoken SGLangAPIOpenAIGPT-4APIAPI

SGLang(LLM)JSONNVIDIA A10GA100 GPULlama-7B/70B [49]Mistral-8x7B [17]LLaVA-v1.5-7B [28]LLaVA-NeXT-34B [62]Guidance [13]vLLM [23]LMQL [4]SGLang6.4×

2

SGLang API

Python Fig. 2 -- [40] `multi_dimensional_judge` `spath` `essay` `s` `path` `essay` SGLang `+=` `s` `select` `s["related"]` fork `gen` `f["judgment"]` JSON `regex` SGLang OpenAI API 2.1×

SGLang Python “`gen`” “`regex`” JSON “`select`” “`+=`” “`extend`” “`[variable_name]`” “`fork`” “`join`” “`image`” “`video`”

SGLang `extendgen` `select` Python CUDA SGLang Appendix D SGLang SGLang Runtime SRT API OpenAI Anthropic

LLM LangChainDSPy LMQLGuidanceSGLang DSPy SGLang LMQL Guidance Table 1 SGLang DSPy SGLang Sec. 6 SGLang DSPy

*Equal contribution.

Preprint. Under review.

```
dimensions = ["Clarity", "Originality", "Evidence"]
@function
def multi_dimensional_judge(s, path, essay):
  s += system("Evaluate an essay about an image.")
  s += user(image(path) + "Essay:" + essay)
  s += assistant("Sure!")

  # Return directly if it is not related
  s += user("Is the essay related to the image?")
  s += assistant(select("related", choices=["yes", "no"]))
  if s["related"] == "no": return

  # Judge multiple dimensions in parallel
  forks = s.fork(len(dimensions))
  for f, dim in zip(forks, dimensions):
    f += user("Evaluate based on the following dimension:" +
      dim + ". End your judgment with the word 'END'")
    f += assistant("Judgment:" + gen("judgment", stop="END"))

  # Merge the judgments
  judgment = "\n".join(f["judgment"] for f in forks)

  # Generate a summary and a grade. Return in the JSON format.
  s += user("Provide the judgment, summary, and a letter grade")
  s += assistant(judgment + "In summary," + gen("summary", stop=".")
                 + "The grade of it is" + gen("grade"))

  schema = r'\{"summary": "[\w\d\s]+\.", "grade": "[ABCD][+-]?"\}'
  s += user("Return in the JSON format.")
  s += assistant(gen("output", regex=schema))

state = multi_dimensional_judge.run(...)
print(state["output"])
```

Handle chat template and multi-modal inputs

Select an option with the highest probability

Fetch result; Use Python control flow

Runtime optimization: KV Cache Reuse (Sec. 3)

Multiple generation calls run in parallel

Fetch generation results

Runtime optimization: API speculative execution (Sec. 5)

Runtime optimization: fast constrained decoding (Sec. 4)

Run an SGLang program

Figure 2: SGLang -- [40]SGLang

Table 1: LMQLGuidance SGLang

LMQL		extend, gen, select	HF Transformers, llama.cpp, OpenAI
Guidance	Python	extend, gen, select, image	HF Transformers, llama.cpp, OpenAI
SGLang	Python	extend, gen, select, image, video, fork, join	SGLang Runtime (SRT), OpenAI

Fig. 2 KV API

3 RadixAttention KV

SGLang "fork" KV LLMKV token [51]KV token KV Appendix A

KV SGLang KV KV [23, 58, 18, 12] KV Sec. 7

RadixAttention KV KV radix tree LRU RadixAttention [60] [23] [44]

RadixAttention token KV KV token GPU KV LRU

token token Fig. 3 CPU fork ""-

$\frac{\text{token}}{\text{token}}$ Alg. 1 [2]

Theorem 3.1. $\geq$

Sec. A.3 DFS DFS Sec. A.3 [42]

RadixAttention GPU GPU KV Sec. A.4

4

LM JSON SGLang regex FSM [54] FSM token token token token token Fig. 2 {"summary": " token Fig. 4 (c) token token FSM token

SGLang FSM FSM Fig. 4 (b) token Fig. 4 (d) token Appendix B

5 API

SGLang API OpenAI GPT-4 API

[2] Theorem 3.1 token KV

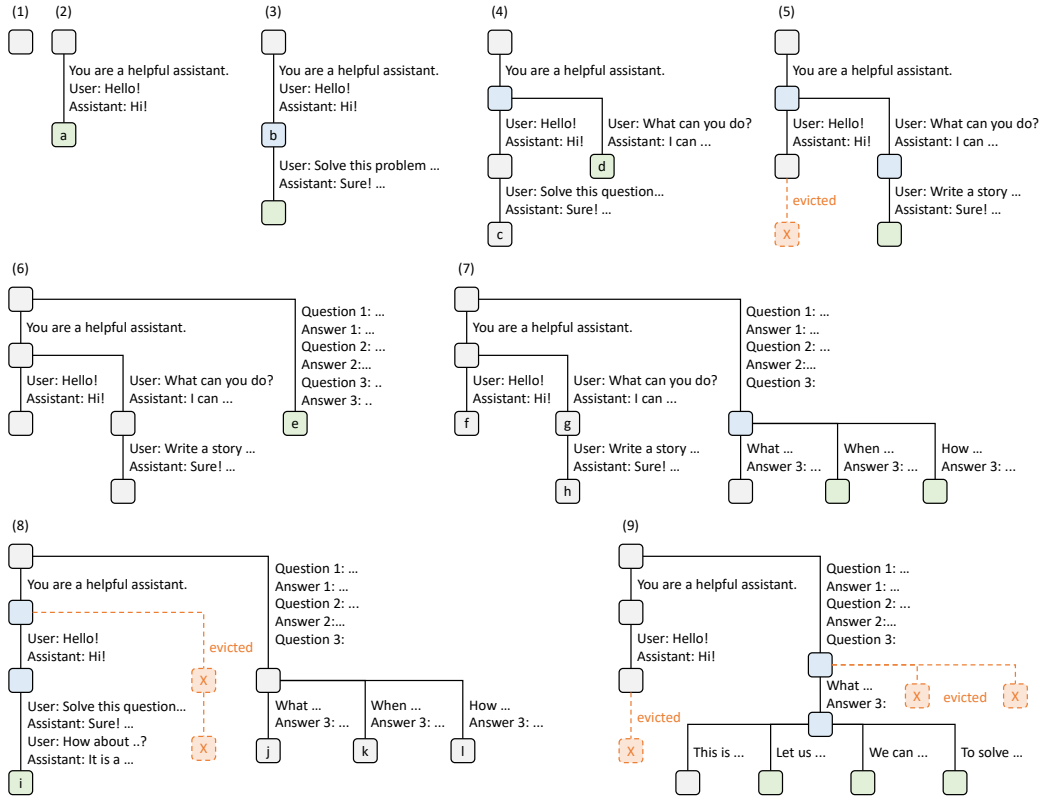


Figure 3: RadixAttention LRU token (1) (2) "Hello" LLM "Hi" "You are a helpful assistant" "Hello!" LLM "Hi!" (3) KV (4) (3) "b" (5) (4) "c" (4) "d" (6) (7) (6) 'e' (8) "g" "h"

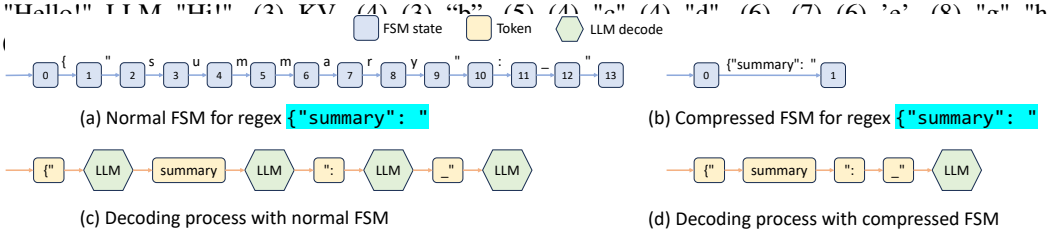


Figure 4: FSM FSM _

API SGLang API `s += context + "name:" + gen("name", stop="\n") + "job:" + gen("job", stop="\n")` `gen` API `context` token SGLang token API

6

(LLM) SGLang SGLang PyTorch [37] FlashInfer [59] CUDA Triton [48]

6.1

Llama-2 [49] Mixtral [17] LLaVA [27] [62] API OpenAI GPT-3.5 70 700 float16

AWS EC2 G5 NVIDIA A10G GPU24GB A10G GPU 7B A10G GPU [44] A100G80GBGPU

SGLang OpenAI Completion API

- Guidance [13](LLM) Guidance v0.1.8 llama.cpp
- vLLM [23] vLLM v0.2.5 API [3]
- LMQL [4] LMQL v0.7.3 Hugging Face Transformers

5-shot MMLU [14] 20-shot HellaSwag [61] MMLU token `select` HellaSwag `select` ReAct [57] [36] GSM-8K (Tree-of-thought) [56] (Skeleton-of-thought) [33] (LLM)--(branch-solve-merge) [40] JSON 4 256-512 token4-8 token256-512 tokenDSPy (RAG) [20]

[3]RadixAttention vLLM

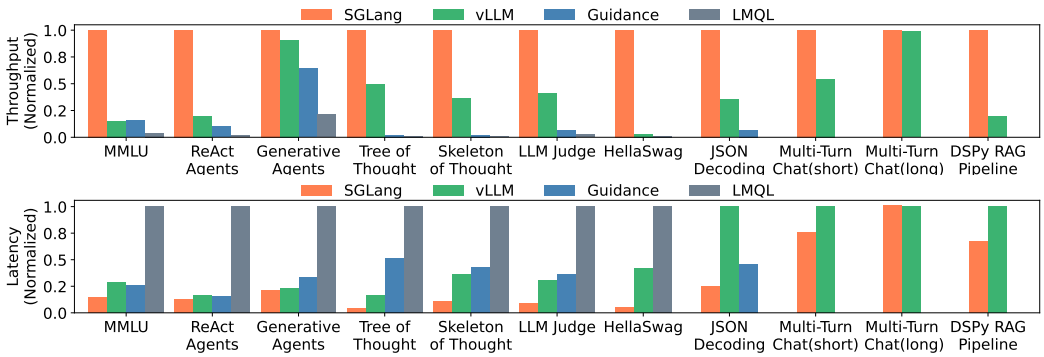


Figure 6: Llama-7B

programs per second, p/s

6.2

Fig. 5 Fig. 6 SGLang 6.4× 3.7× KV

MMLU SGLang RadixAttention 5-shot KV RadixAttention RadixAttention KV RadixAttention (prefill) token HellaSwag SGLang few-shot KV ReAct SGLang KV SGLang KV JSON SGLang token SGLang KV KV DSPy RAG SGLang KV 50% 99% Fig. 13 96%

LMQL Guidance LMQL token Guidance

Mixtral-8x7B Llama-70B Fig. 7 Fig. 12 Guidance LMQL

SGLang `image` `video` RadixAttention(radix tree) token KV llava-bench-in-the-wild LLaVA-v1.5-7B ActivityNet LLaVA-NeXT-34B Hugging Face Transformers Table 2 SGLang 6× llava-bench-in-the-wild SGLang KV

SGLang Chatbot Arena [8] SGLang worker LLaVA-Next-34B [28] RadixAttention 52.4%Vicuna-33B [7] 74.1% Vicuna-33B token 1.7×

API OpenAI GPT-3.5 few-shot API (speculative execution) token

6.3

/ Fig. 8(a)(b) token token

RadixAttention RadixAttention Fig. 8(c) "No Cache" "No Tree-Structure" "FCFS Schedule" "Random Schedule" "No Frontend Parallelism" "No Frontend Hint" (fork hints)"Full optimizations"

RadixAttention KV RadixAttention ShareGPT 100 74.3 RadixAttention 0.2 0.3% RadixAttention

JSON 1.6× token 2.4×

7

KV RadixAttention KV LRU - vLLM [23] ChunkedAttention [58] LRU PromptCache [12] KV 43%HydraGen [18]FlashInfer [59] ChunkedAttention [58] CUDA LRU API Serve [1] LLM-SQL [29] KV API (radix tree)

(LLM) Guidance [13]LMQL [4]DSPy [20]LangChain [24]AutoGen [55] LLM Compiler [21]Guidance LMQL SGLang Sec. 2 SGLang DSPy SGLang [60, 39, 3, 23, 59, 10, 26, 15, 19, 32, 31, 11]

8

SGLangSGLangRadixAttentionDRAM [43]RadixAttentionSGLang [42] SGLang

SGLangSGLangRadixAttentionLM

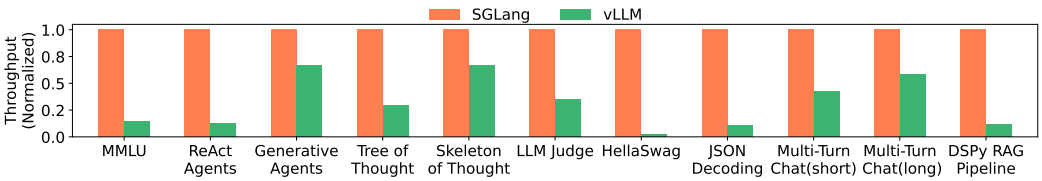


Figure 7: Mixtral-8x7B

Table 2: LLaVA

	LLaVA-v1.5-7B	LLaVA-NeXT-34B
SGLang	0.18 / **1.15 /**	0.02 / **0.10 /**

AstronomerGoogleIBMIntelLaceworkMicrosoftMohamed Bin Zayed University of Artificial IntelligenceNexlaSamsung SDSUberVMware Lianmin ZhengMeta Yuanhan ZhangBo LiLLaVA-NeXT

References

[1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiying Zhang. Apiserve: Efficient api support for large-language model inferencing. *arXiv preprint arXiv:2402.01869*, 2024.

[2] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. *Advances in Neural Information Processing Systems*, 35:23716–23736, 2022.

[3] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE, 2022.

[4] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969, 2023.

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[6] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.

[7] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023.

[8] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E Gonzalez, et al. Chatbot arena: An open platform for evaluating llms by human preference. *arXiv preprint arXiv:2403.04132*, 2024.

[9] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.

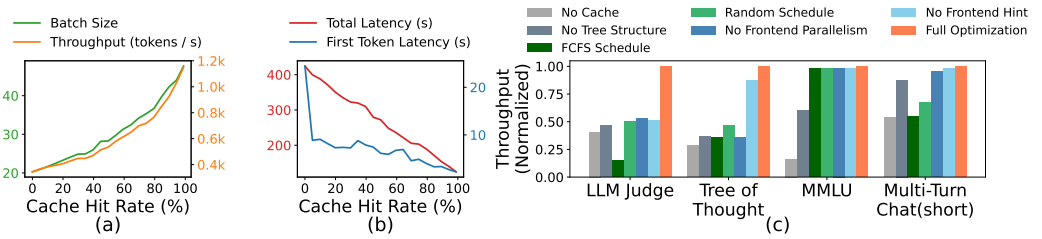


Figure 8: (a)(b) (c) RadixAttention

[10] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.

[11] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms. In *The Twelfth International Conference on Learning Representations*, 2023.

[12] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.

[13] Guidance AI. A guidance language for controlling large language models. https://github.com/guidance-ai/guidance. Accessed: 2023-11.

[14] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2020.

[15] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

[16] Hugging Face. Text generation inference. https://github.com/huggingface/text-generation-inference. Accessed: 2023-11.

[17] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

[18] Jordan Juravsky, Bradley Brown, Ryan Ehrlich, Daniel Y Fu, Christopher Ré, and Azalia Mirhoseini. Hydragen: High-throughput llm inference with shared prefixes. *arXiv preprint arXiv:2402.05099*, 2024.

[19] Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. Gear: An efficient kv cache compression recipefor near-lossless generative inference of llm. *arXiv preprint arXiv:2403.05527*, 2024.

[20] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

[21] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling. *arXiv preprint arXiv:2312.04511*, 2023.

[22] Michael Kuchnik, Virginia Smith, and George Amvrosiadis. Validating large language models with relm. *Proceedings of Machine Learning and Systems*, 5, 2023.

[23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[24] LangChain AI. Langchain. https://github.com/langchain-ai/langchain. Accessed: 2023-11.

[25] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.

[26] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

[27] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. *arXiv preprint arXiv:2310.03744*, 2023.

[28] Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, January 2024.

[29] Shu Liu, Asim Biswal, Audrey Cheng, Xiangxi Mo, Shiyi Cao, Joseph E Gonzalez, Ion Stoica, and Matei Zaharia. Optimizing llm queries in relational workloads. *arXiv preprint arXiv:2403.05821*, 2024.

[30] Xiaoxia Liu, Jingyi Wang, Jun Sun, Xiaohan Yuan, Guoliang Dong, Peng Di, Wenhai Wang, and Dongxia Wang. Prompting frameworks for large language models: A survey. *arXiv preprint arXiv:2311.12785*, 2023.

[31] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems*, 36, 2024.

[32] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.

[33] Xuefei Ning, Zinan Lin, Zixuan Zhou, Zifu Wang, Huazhong Yang, and Yu Wang. Skeleton-of-thought: Prompting LLMs for efficient parallel generation. In *The Twelfth International Conference on Learning Representations*, 2024.

[34] NVIDIA. Tensorrt-llm. https://github.com/NVIDIA/TensorRT-LLM. Accessed: 2023-11.

[35] OpenAI. Gpt-4 technical report, 2023.

[36] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *In the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, UIST '23, New York, NY, USA, 2023. Association for Computing Machinery.

[37] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[38] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023.

[39] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5, 2023.

[40] Swarnadeep Saha, Omer Levy, Asli Celikyilmaz, Mohit Bansal, Jason Weston, and Xian Li. Branch-solve-merge improves large language model evaluation and generation. *arXiv preprint arXiv:2310.15123*, 2023.

[41] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.

[42] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E Gonzalez, and Ion Stoica. Fairness in serving large language models. *arXiv preprint arXiv:2401.00588*, 2023.

[43] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: high-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*, pages 31094–31116. PMLR, 2023.

[44] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[45] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiying Zhang. Preble: Efficient distributed prompt scheduling for llm serving. 2024.

[46] Theodore Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2023.

[47] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[48] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

[49] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[50] Vivien Tran-Thien. Fast, high-fidelity llm decoding with regex constraints, 2024.

[51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[52] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv: Arxiv-2305.16291*, 2023.

[53] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2022.

[54] Brandon T Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv e-prints*, pages arXiv–2307, 2023.

[55] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.

[56] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*, 2023.

[57] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2022.

[58] Lu Ye, Ze Tao, Yong Huang, and Yang Li. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition. *arXiv preprint arXiv:2402.15220*, 2024.

[59] Zihao Ye, Lequn Chen, Ruihang Lai, Yilong Zhao, Size Zheng, Junru Shao, Bohan Hou, Hongyi Jin, Yifei Zuo, Liangsheng Yin, Tianqi Chen, and Luis Ceze. Accelerating self-attentions for llm serving with flashinfer, February 2024.

[60] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.

[61] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, 2019.

[62] Yuanhan Zhang, Bo Li, haotian Liu, Yong jae Lee, Liangke Gui, Di Fu, Jiashi Feng, Ziwei Liu, and Chunyuan Li. Llava-next: A strong zero-shot video understanding model, April 2024.

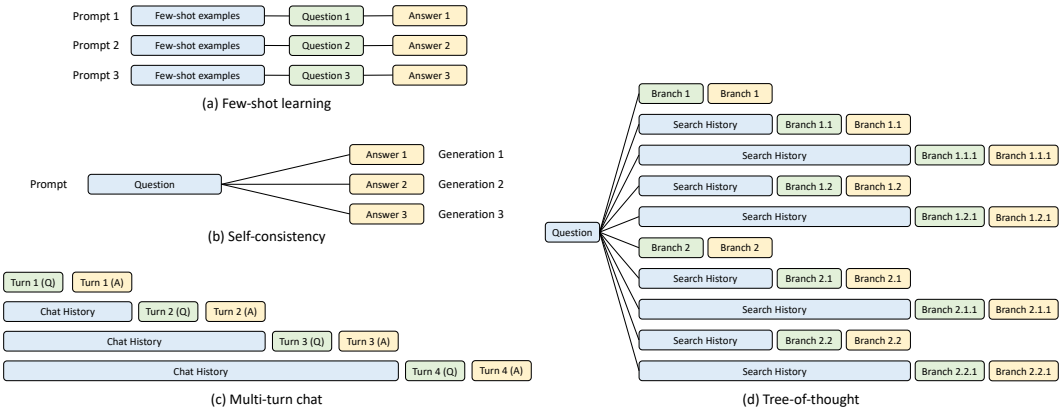


Figure 9: KV self-consistency [53] tree-of-thought [56]

A RadixAttention

A.1 KV

LLM GPT-3 [5]PaLM [9] LLaMA [49] Transformer [51] "" "" "KV " KV KV
LLM KV LLM [25]vLLM [23] Fig. 9 KV Sec. 3 RadixAttention

A.2

Alg. 1 RadixAttention

A.3 Theorem 3.1

Theorem 3.1. $\geq$

Proof. (DFS) R T R T e e KV $|e|$ e KV C R KV

$$C \geq \sum_{e \in \text{edges}(T)} |e|.$$

DFS T T e e KV e e e e e $\geq$ T e e e KV

$$C = \sum_{e \in \text{edges}(T)} |e|.$$

$$\frac{\sum_{r \in R} r}{\sum_{r \in R} r},$$

$1 - \frac{C}{\sum_{r \in R}}$
DFS

- T x x $\{v_1, ..., v_n\}$ $\{v_1, ..., v_n, x\}$ DFS x
- T y DFS P y P P P z z DFS z

□

DFS DFS
T T' C T longest(C) C T'
DFS longest(C) T' C $C^{(1)} \subseteq C$ longest$(C^{(1)})$ T' DFS
$C^{(2)}, \ldots, C^{(k)}$ $C^{(k)}$ $C^{(k)}$ T' DFS T' Theorem 3.1 DFS

Algorithm 1 RadixAttention

Input: T P B Q
Output:
  //
  $requests \leftarrow Q$.get_all_requests()
  //
  for $req \in requests$ **do**
    $req.prefix\_node, req.prefix\_len \leftarrow T$.match_prefix$(req.input_tokens)$
  end for
  //
  $requests$.sort()
  //
  $available\_size \leftarrow T$.evictable_size() $+ P$.available_size()
  $current\_size \leftarrow 0$
  $new\_batch \leftarrow []$
  for $req \in requests$ **do**
    if req.size() $+ current\_size < available\_size$ **then**
      new_batch.append(req)
      $delta \leftarrow T$.increase_ref_counter$(req.prefix_node)$
      $available\_size \leftarrow available\_size + delta$
    end if
  end for
  Q.remove_requests(new_batch)
  //
  B.merge(new_batch)
  //
  $needed\_size \leftarrow B$.needed_size()
  $success, buffer \leftarrow P$.alloc$(needed_size)$
  if not $success$ **then**
    T.evict$(needed_size)$
    $success, buffer \leftarrow P$.alloc$(needed_size)$
  end if
  B.run$(buffer)$
  //
  $finished\_requests \leftarrow B$.drop_finished_requests()
  for $req \in finished\_requests$ **do**
    T.decrease_ref_counter$(req.prefix_node)$
    T.insert(req)
  end for
  return $finished\_requests$

A.4 RadixAttention

RadixAttention trie MMLU Preble [45] SGLang

B

LLMregex JSON FSM [54]FSM /LLM FSM Fig. 10 FSM /LLM [22]

[4]

B.1

FSM / FSM

[4] https://lmsys.org/blog/2024-02-05-compressed-fsm/

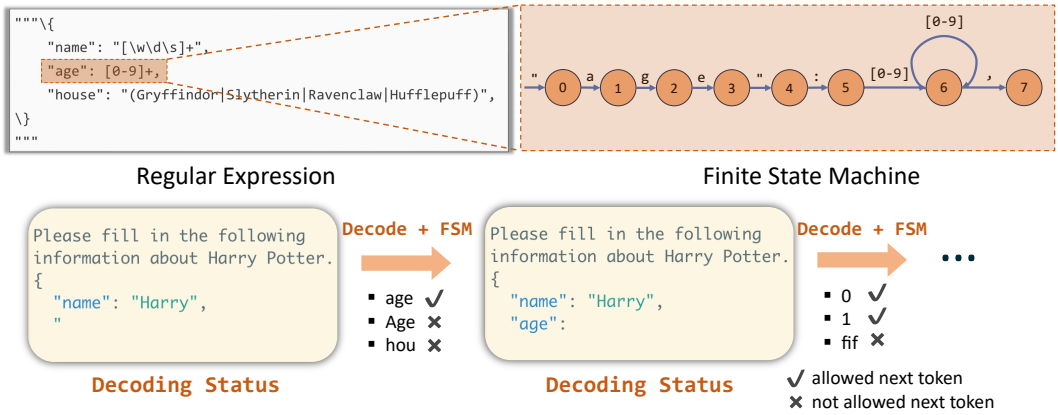


Figure 10: FSM FSM

- 1) 2) /
- $(e_0, e_1, \ldots, e_k)$ $e_1, \ldots, e_k$ $e_0, e_1, \ldots, e_k$

FSM FSM SGLang FSM Fig. 11

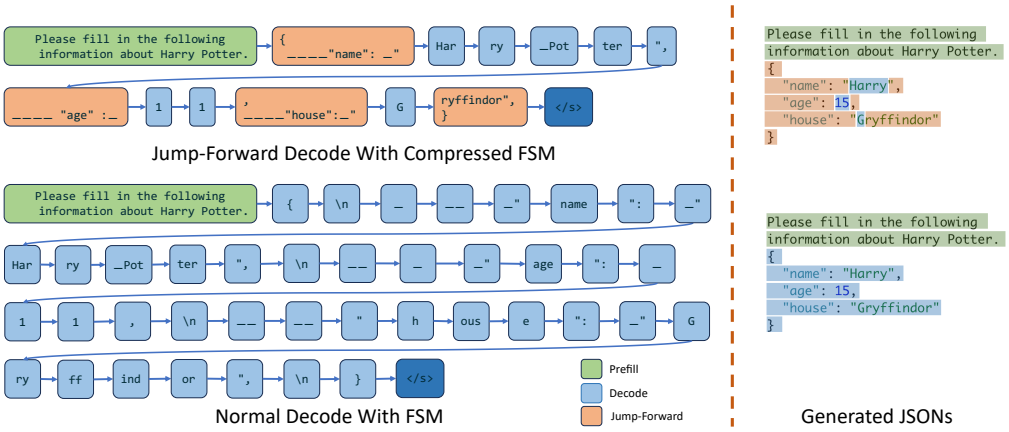


Figure 11: FSM FSM

B.2

LLM [50]Fig. 2 `{"summary": "{", summary, ": _"{", summa, ry ":_"`LLM

B.3

[50] Fig. 2 `"[ABCD][+-]?" A+ D- Excellent|Above Average|Fair|Below Average Above Average A Above Average`LLMLLM FSM

C

Fig. 5 Fig. 6 A10G (24GB) GPU Llama-7B Fig. 7 8 A10G (24GB) GPU Mixtral-8x7B Fig. 8(c) A10G (24GB) GPU Llama-7B Fig. 12 4 A100G (80GB) GPU Llama-70B Table 2 A10G (24GB) GPU LLaVA-v1.5-7B A100G (80GB) GPU LLaVA-Next-34B

Fig. 13 Fig. 5 Fig. 12 Llama-2-70B

D

SGLang

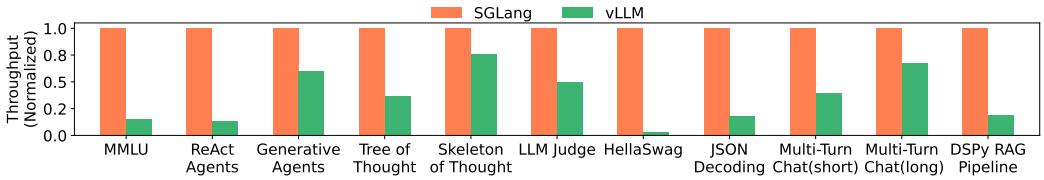


Figure 12: Llama-2-70B

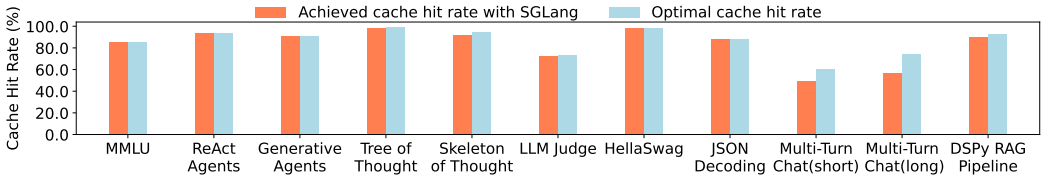


Figure 13:

D.1

SGLang IR SGLang Fig. 14b Fig. 14a fork

SGLang `+=` `+` IR `ConstantTextArgumentGenSelectVariableForkGetForkItem` `Join` `+=` fork

Python IR

D.2

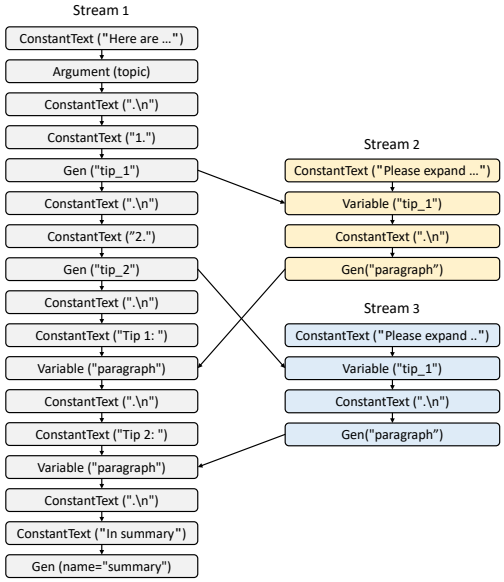
SGLang IR

“Here is a question + {question}. Please act as a math expert and solve the given question.”“Please act as a math expert and solve the given question. Here is a question + {question}.” SGLang GPT-4 GPT-4 SGLang IR GPT-4 SGLang

20 SGLang 5 15 15 12 GPT-4 60 GPT-4 GPT-4

```
@function
def expand(s, tip):
    s += (
        "Please expand the following tip into a "
        "detailed paragraph: " + tip + "\n"
    )
    s += gen("paragraph")

@function
def tip_suggestion(s, topic):
    s += "Here are 2 concise tips for " + topic + ".\n"

    # Generate a skeleton of two short tips
    s += "1." + gen("tip_1", stop=["\n", ":", "."]) + "\n"
    s += "2." + gen("tip_2", stop=["\n", ":", "."]) + "\n"

    # Expand the tips into detailed paragraphs in parallel
    detailed_tip1 = expand(tip=s["tip_1"])
    detailed_tip2 = expand(tip=s["tip_2"])

    # Merge the results and generate a summary
    s += "Tip 1: " + detailed_tip1["paragraph"] + "\n"
    s += "Tip 2: " + detailed_tip2["paragraph"] + "\n"
    s += "In summary" + gen("summary")

state = tip_suggestion.run(topic="staying healthy")
print(state.text())
```

(a) skeleton-of-thought SGLang



(b) Fig. 14a

Figure 14: SGLang